

System Architecture & Design Document

Architecture, module design, data model, and the algorithms governing matching, escalation, and revenue division

DOCUMENT REFERENCE	CB-SAD-2026-003
VERSION	1.0
DATE OF ISSUE	2026-07-18
PREPARED FOR	CareBridge Health
PREPARED BY	CareBridge Engineering Team
CLASSIFICATION	Confidential

Table of Contents

1 Introduction	3
1.1 Purpose and Audience	3
1.2 Design Principles	3
2 System Architecture	4
2.1 Architectural Overview	4
2.2 Layer Responsibilities	5
2.3 Request Path	5
2.4 Client Architecture	6
3 Data Model	7
3.1 Entity Relationships	7
3.2 The Identity Model	9
3.3 Data Dictionary — Core Entities	9
3.4 Supporting Entities	12
3.5 Indexing Strategy	13
4 The Matching Engine	14
4.1 Purpose	14
4.2 Hard Filters	14
4.3 Scoring Factors	14
5 Referral Lifecycle and Escalation	16
5.1 State Machine	16
5.2 Response Deadlines	16
5.3 The Escalation Job	17
6 The Commission Engine	18
6.1 Why It Is a Single Module	18
6.2 The Two Models	18
6.3 The Governing Invariant	19
6.4 Terms Resolution Order	19
6.5 The Laboratory Variant	19
6.6 Precision and Guards	20
7 The Settlement Cycle	21
7.1 Design Intent	21
7.2 The Five States	21
7.3 Accrual and Aggregation	21
8 Cross-Cutting Design	22
8.1 Authentication and Authorisation	22
8.2 Field-Level Encryption	22
8.3 Notifications	22
8.4 Real-Time Event Model	23
8.5 Configuration and White-Labeling	23
9 Known Design Debt	24

1. Introduction

1.1 Purpose and Audience

This document describes how the CareBridge platform is built. It is written for the engineers who will maintain and extend the system, and for the technical reviewer assessing it at handover. It covers the architecture, the responsibilities of each layer, the complete data model, and — in detail — the three algorithms that determine the system's behaviour: hospital matching, referral escalation, and revenue division.

1.2 Design Principles

Five principles governed the design. They are stated here because they explain decisions that would otherwise look arbitrary in the sections that follow.

- **One calculation path for money** — every division of a bill goes through one module. The original design record identifies bill-splitting arithmetic duplicated across four separate locations as the principal defect risk in this system; consolidating it was treated as a correctness requirement, not a refactor.
- **Integer money** — every monetary value is a whole number of paisa. No monetary value is ever held as a floating-point number, in the database, in transit, or in calculation.
- **Snapshot what was applied** — commercial terms change. A payout record therefore carries the model, type, rate, and resulting amounts that were used, so a figure computed a year ago can still be explained.
- **Additive change to live commercial terms** — the additive commission model was introduced while the platform was in use. Every existing party defaults to the previous model and is opted in individually, so adoption changes nobody's terms until an administrator acts.
- **Push, do not poll** — inboxes, dashboards, and bed counts are time-sensitive. State changes are pushed to the affected clients over a persistent connection, with a slow interval refresh as a fallback only.

2. System Architecture

2.1 Architectural Overview

CareBridge is a two-tier web application: a single-page client and a stateless API server, with all persistent state in a document database and all binary assets in an external file store. The client and server are deployed independently on separate managed platforms and communicate over two channels — a versioned REST interface for request-response, and a persistent socket connection for server-initiated updates.



Figure 1: Deployment and component architecture

2.2 Layer Responsibilities

Layer	Responsibility	Must not
Routes	Declare the path, attach the authentication and role guards, and name the controller function.	Contain business logic or touch the database.
Controllers	Validate and normalise input, orchestrate calls to services and models, and shape the HTTP response.	Perform financial arithmetic. Bill splitting belongs in the commission engine without exception.
Services	Hold business logic. The commission engine is pure and has no database access at all, which is what makes it exhaustively testable.	Depend on the HTTP request or response objects.
Models	Define schema, validation, indexes, and field-level encryption hooks.	Contain cross-entity workflow logic.
Socket registry	Hold the single server reference so that services and scheduled jobs can emit events without an HTTP request in scope.	Be constructed more than once.

2.3 Request Path

A representative authenticated request traverses the stack as follows. Understanding this path is sufficient to locate any behaviour in the codebase.

```

Browser
| PATCH /v1/referrals/:id/status   Authorization: Bearer <token>
v
helmet ..... protective response headers
cors ..... origin checked against the configured allowlist
express.json ... body parsed
|
v
protect ..... token verified; identity attached to the request
authorize ..... role checked (administrators pass unconditionally)
|
v
referralRoutes ... path matched
|
v
referralController.updateStatus
|   validate transition, load referral, apply change
+--> Referral model ..... persist
+--> notificationService ..... in-app + email + WhatsApp fan-out
+--> socket registry ..... emit STATUS_UPDATE to the affected rooms
|
v
JSON response

```

Figure 2: Path of an authenticated request

2.4 Client Architecture

The client is a single React application serving all four roles. Role separation is achieved by routing and guards rather than by separate builds, so shared behaviour — authentication, branding, notifications, the socket connection — is implemented once.

Concern	Mechanism
Routing and role guards	A single route table with a guard component that redirects a user requesting a screen outside their role. Route paths are namespaced by role.
Server state	A query cache holding fetched data with a staleness window, so repeated navigation does not re-request unchanged data.
Authentication	A context provider holding the session, decoding the identity from the token, and scheduling a renewal shortly before expiry so that an active user is never interrupted.
Request handling	A configured HTTP client that attaches the credential to every request and, on rejection, renews the session once and replays the original request. Concurrent rejections queue behind a single renewal rather than triggering several.
Real-time updates	One socket connection for the session. Incoming events are debounced and collapsed into a single cache invalidation, so a burst of events causes one refresh rather than many.
Branding	Platform and hospital branding are resolved into an effective set and applied as CSS custom properties at runtime, which is why brand-coloured elements use inline styles rather than static classes.

TWO KNOWN CLIENT-SIDE INCONSISTENCIES

First, screen-level data fetching is mixed: newer screens use the query cache, while several older screens fetch directly in an effect. Both work; the difference is a maintenance inconsistency rather than a defect. Second, two screens open their own socket connection in addition to the shared one, a pattern that predates the shared connection. Both are recorded in the Maintenance and Support Guide as consolidation candidates.

3. Data Model

3.1 Entity Relationships

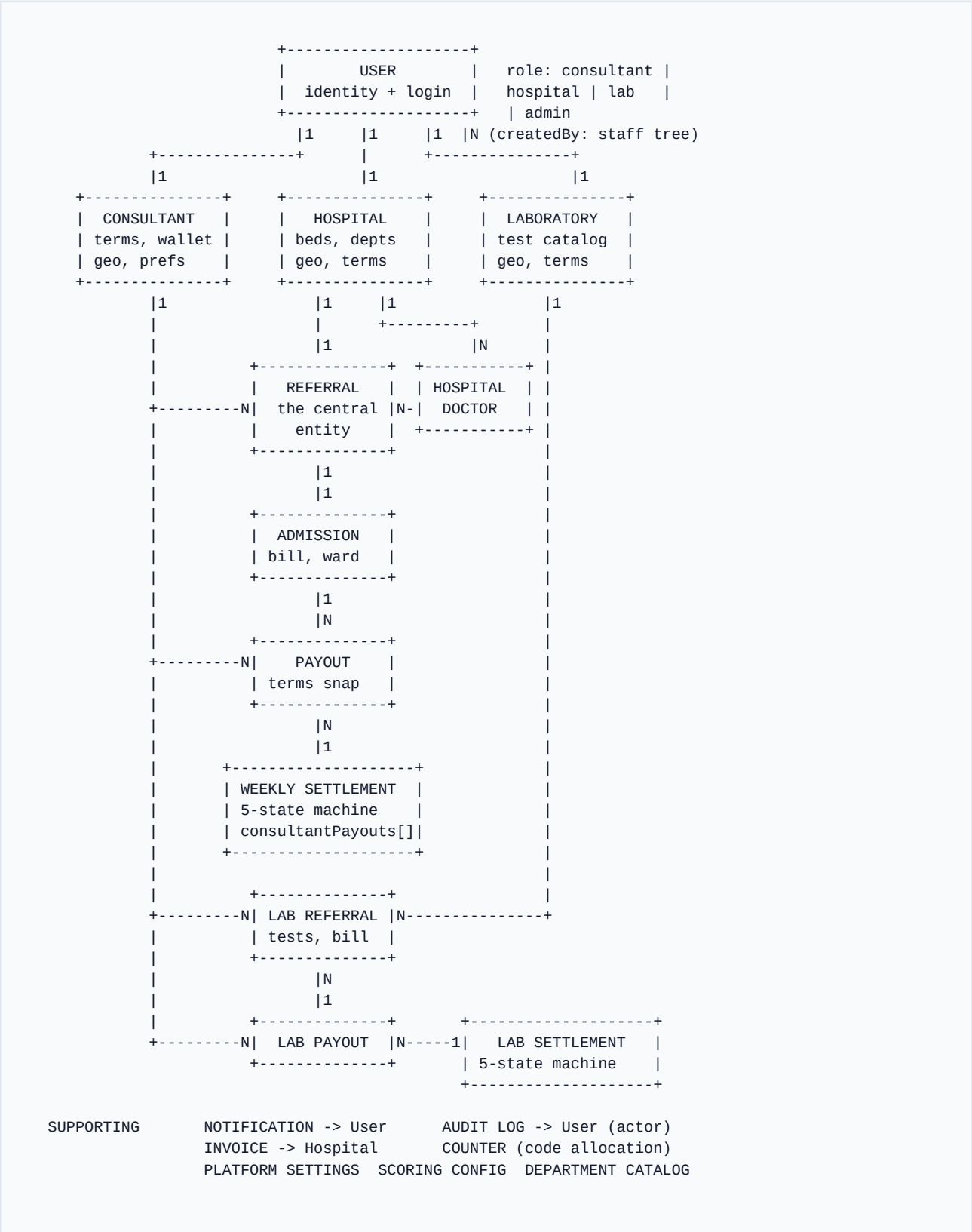


Figure 3: Entity relationship overview

3.2 The Identity Model

One collection holds every login, whatever the role. A user is linked to a facility in one of two ways, and the distinction governs the whole permission model.

- **Owner account** — the facility record points at the user. This is the account created at registration and it owns the organisation.
- **Team member account** — the user record points at the facility. These accounts are created by the owner and have no facility record of their own.
- **Creation chain** — every account records who created it. Deletion of a team member is authorised by walking this chain, which prevents a member from removing an account above their own.

A CONSEQUENCE WORTH STATING EXPLICITLY

Owner accounts and team member accounts currently carry the same permissions within their facility. The distinction exists in the data model and is used for deletion authorisation, but not for feature-level permission. Should a facility-level administrator tier with elevated privilege be required in future, this is the model it would build on.

3.3 Data Dictionary — Core Entities

3.3.1 User

Field	Type	Description
role	Enumeration, required	consultant, hospital, admin, or laboratory
name, phone	String, required	Contact identity
email	String, required, unique	Sign-in identifier, stored lowercase
passwordHash	String, required	Salted hash; the plaintext password is never stored
hospitalId, labId	Reference, nullable, indexed	Set on team member accounts only, identifying their facility
createdBy	Self-reference, nullable	The account that created this one; forms the creation chain
status	Enumeration	pending, active, or suspended. Only active accounts may sign in.
isEmailVerified	Boolean	Sign-in is refused until true
emailVerificationToken / Expires	String / Date	Legacy link-based verification path
resetPasswordToken / Expires	String / Date	Single-use, time-limited password reset credential

3.3.2 Consultant

Field	Type	Description
userId	Reference, unique	The owning login
pmdcNumber	String, unique	Professional registration number
cnic, specialty, clinicName, clinicAddress, city	String	Professional profile; specialty is required
location	Geographic point, indexed	Clinic coordinates, used for distance scoring
preferredHospitals	Reference array	Explicitly favoured hospitals, feeding the preference score
referralHistoryCount	Map of hospital to count	Learned preference, incremented as referrals are sent
walletBalance, totalEarnings, monthlyEarnings	Integer paisa	Earnings position
verificationDocuments	Array of name, URL, timestamp	Uploaded certificates and identity documents
commissionModel	Enumeration	legacy or additive; defaults to legacy so existing terms are unchanged
commissionPercentage	Number	Share of the platform deduction under the legacy model
hospitalCommissionType / Percentage / FixedPaisa	Enumeration / Number / Integer	Additive-model consultant terms for hospital referrals, charged per referral
labCommissionType / Percentage / FixedPaisaPerTest	Enumeration / Number / Integer	Additive-model consultant terms for laboratory referrals, charged per test
facilityPlatformOverrides	Embedded array	Per-consultant, per-facility platform charge, overriding the facility's own terms
maxLabDiscountPercentage	Number	Ceiling on the discount this consultant may grant on a laboratory referral
payoutAccount	Embedded object	Payout destination — mobile wallet or bank

3.3.3 Hospital and Laboratory

These two entities are structurally parallel. Both hold identity, verification documents, geographic location, branding, commercial terms, and payment gateway credentials. They differ in their operational inventory and in the unit their fixed platform charge applies to.

Field	Hospital	Laboratory
Name field	hospitalName	labName

Field	Hospital	Laboratory
Operational inventory	bedsInventory — ward, total, occupied, and available beds across ten ward types	testCatalog — test name, price in paisa, and turnaround hours
Service definition	departments — determines referral eligibility	testCatalog entries serve the same purpose
Pricing reference	ratePackages — department, service, and a minimum and maximum price, used for cost-fit scoring	Not applicable; test prices serve directly
Performance record	avgResponseTime, acceptanceRate, rating — feed the response-history score	Not scored on response history
Platform charge type	percentage or fixed	percentage or fixed
Fixed charge unit	fixedPlatformChargePaisa — charged once per referral	fixedPlatformChargePaisaPerTest — charged per test line
Activation	isActive, default false — a hospital is not matchable until activated	isActive, default false

THE FIXED-CHARGE UNIT DIFFERS BETWEEN THE TWO

A hospital fixed charge applies once per referral. A laboratory fixed charge applies once per test, so a referral ordering five tests incurs it five times. This is deliberate and reflects how the two businesses price, but it is the single easiest thing to misread in the commercial model. The field names differ specifically to make the distinction visible in code.

3.3.4 Referral

Group	Fields
Identity	referralCode — unique, of the form CB-YYYY-NNNN, allocated atomically; consultantId
Patient	patientName, dateOfBirth (the source of truth for age), age (derived), gender, phone, area, cnic (encrypted at field level), guardianName, guardianRelation
Clinical	urgency, symptomsText, summaryNotes, symptomTags, department, assignedDepartment, diagnosisText, notes, attachments
Clinical notes	An embedded timeline of entries, each with a type of nursing or consultant, content, author reference, author name, and timestamp
Budget	budgetMin, budgetMax in paisa, and a budget bracket, used for cost-fit scoring
Routing	targetHospitalId — the hospital currently responsible; targetDoctorId; rankedHospitalIds — the escalation queue; rankedHospitalPreferences — the department and doctor chosen for each ranked hospital; currentRankIndex — position in the queue

Group	Fields
State	status — pending, accepted, rejected, admitted, or closed; rejectionReason; scoringData — a snapshot of the ranking basis; slaDeadline — the current response deadline
Timeline	acceptedAt, admittedAt, closedAt

3.3.5 Admission

Field	Type	Description
referralId	Reference, unique	Enforces exactly one admission per referral
hospitalId, consultantId	Reference, required	The parties to the case
status	Enumeration	active, discharged, or billed
roomNumber, bedNumber, admissionDepartment, treatingDoctorId	Required	Physical and clinical placement, all mandatory at admission
services	Embedded array	Bill lines of description and amount in paisa
billTotalPaisa	Integer paisa	Sum of the service lines
paymentMethod, paymentReference	Enumeration, String	How the patient settled with the hospital
patientBillFileUrl	String	The attached patient bill document
weeklySettlementId	Reference, nullable	Set when the admission is drawn into a settlement
admitDate, dischargeDate, completedAt	Date	Case timeline

3.3.6 Payout and Settlement

A payout is the record of one consultant's commission on one billed case. Beyond the amount, it carries a complete snapshot of the terms that produced it — the model, both component types, both rates, both fixed amounts, and the resulting consultant commission, platform charge, and total. This is what makes a historical figure explainable after terms have changed, and it is the reason the record has as many fields as it does.

A settlement bundles billed cases for one facility over a stated period. It records the aggregate amounts, the uploaded documents at each stage, the verifying administrator, and an embedded list of consultant payouts each with its own status and receipt. The laboratory settlement is an exact structural mirror.

3.4 Supporting Entities

Entity	Purpose
Counter	Atomic sequence allocation for referral codes, keyed by type and year, guaranteeing uniqueness under concurrency.
PlatformSettings	Platform-wide defaults — commission rates, wallet thresholds, and branding. Read as the most recently updated record.
ScoringConfig	The six matching weights. The schema refuses to save a configuration whose weights do not total 100, so an invalid configuration cannot reach the engine.
DepartmentCatalog	Department names and the symptom keywords that map to them, driving automatic department suggestion.
Notification	Per-user notification records with a read flag, indexed for the unread count query. Personal data is stripped before storage.
AuditLog	Administrative action records — actor, action, affected record and its type, network address, and client.
Invoice	Platform invoices raised against hospitals, separate from the settlement cycle.
HospitalDoctor	Doctors employed by a hospital. Distinct from consultants, who are the external referring doctors.

3.5 Indexing Strategy

Index	Supports
Geographic index on consultant, hospital, and laboratory location	Distance scoring and the emergency proximity fallback, which performs a nearest-neighbour query with a radius bound.
Hospital and laboratory reference on user	Resolving a facility's team members for notification fan-out and team management.
Hospital and specialty on hospital doctor	Doctor lookup when a consultant selects a named doctor at a hospital.
Consultant with creation time, descending, on payout	The consultant earnings view, which is read frequently and ordered by recency.
Hospital with period start, descending, on settlement	The facility settlement history view.
User, read flag, and creation time on notification	The unread count query, which runs on every page load for every user.
Unique index on referral code, professional registration number, and email	Uniqueness guarantees at the database level rather than by application check alone.

4. The Matching Engine

4.1 Purpose

When a consultant requests recommendations, every candidate hospital is scored against the referral and returned in ranked order. The ranking is also persisted on the referral as the escalation queue, so the same ordering governs automatic reassignment if the first hospital does not respond.

4.2 Hard Filters

Two conditions eliminate a hospital outright, before any scoring. A hospital failing either is not ranked low — it is excluded, because recommending it would waste the response deadline on a hospital that cannot take the patient.

- **Department** — the hospital must operate the department the referral requires.
- **Beds** — the hospital must have at least one available bed in the required ward. The required ward is intensive care for an emergency referral and a general ward otherwise.

4.3 Scoring Factors

Factor	Default weight	Calculation
Department match	30	Awarded in full when the department is operated. A hospital reaching the scoring stage has already passed this filter, so in practice this weight is a constant that sets the floor for a viable hospital.
Bed availability	25	Scaled by the proportion of beds free in the required ward. A hospital with one bed of forty free scores far below one with twenty of forty free, which spreads load rather than saturating the nearest hospital.
Distance	15	Great-circle distance from the patient, scoring linearly from full weight at zero distance to nothing at a thirty kilometre horizon.
Cost fit	10	Full weight where the hospital's maximum price for the service falls within the referral budget; half weight where only its minimum price does.
Response history	10	Combines average response time and acceptance rate. A hospital averaging a one-hour response scores below one averaging fifteen minutes. This is what makes prompt response commercially rational for a hospital.
Consultant preference	10	Full weight for an explicitly favoured hospital; otherwise scaled by how many referrals this consultant has previously sent there, capped at the weight.

The six weights are administrator-configurable and must total 100; the schema refuses to save any other configuration. The final score is capped at 100. The scoring basis is snapshotted onto the referral so a past recommendation remains explainable after weights change.

```
referral (department, ward, urgency, budget, patient location)
|
v
for each active hospital:
|
+--> operates the department? -- no --> EXCLUDE
|
+--> bed free in required ward? -- no --> EXCLUDE
|
+--> yes (ICU if emergency, else General)
|
v
score = departmentMatch (w=30, full)
+ bedAvailability (w=25 x free/total)
+ distance (w=15 x max(0, 1 - km/30))
+ costFit (w=10 full | half | zero)
+ responseHistory (w=10 x f(avgResponseTime, acceptanceRate))
+ preference (w=10 full if favourite, else min(w, history x 2))
|
v
cap at 100 -> sort descending -> ranked list
|
+--> returned to the consultant as recommendations
+--> persisted on the referral as the escalation queue
```

Figure 4: Scoring flow for one referral

5. Referral Lifecycle and Escalation

5.1 State Machine

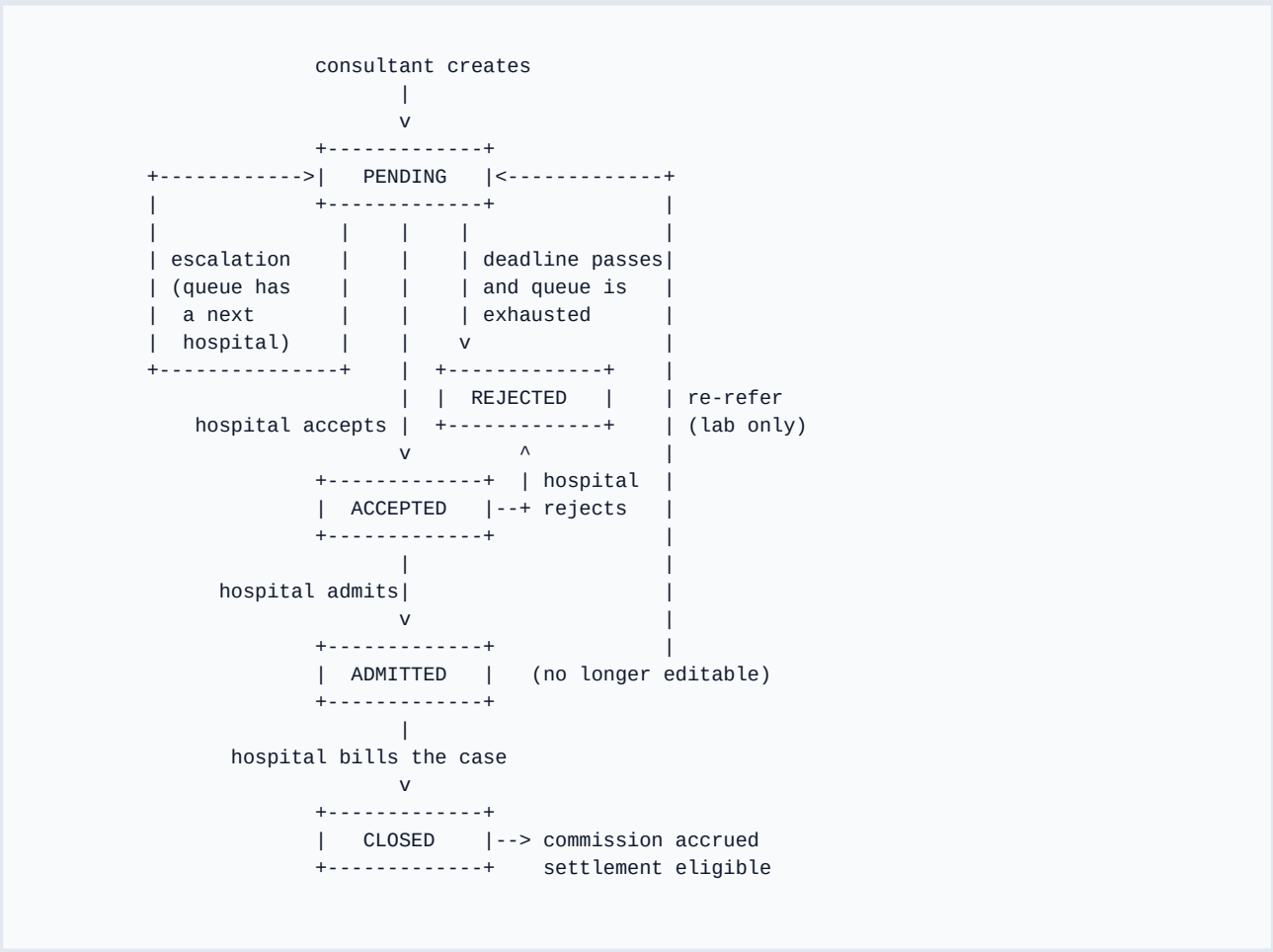


Figure 5: Referral states and transitions

5.2 Response Deadlines

Urgency	Deadline	Rationale
Emergency	15 minutes	A patient requiring immediate care cannot wait on an unanswered referral. Emergencies also have a geographic fallback beyond the ranked queue.
Urgent	2 hours	Same-day treatment; long enough for a hospital to check capacity, short enough to preserve the day.
Routine	24 hours	Elective referral; a full working day for the hospital to respond.

5.3 The Escalation Job

A scheduled job runs every minute, selecting referrals that are still pending and whose deadline has passed. For each, it applies the following logic in order.

1. **If the ranked queue holds a further hospital** — advance the queue position, reassign the referral, apply the department and doctor the consultant chose for that hospital, and set a fresh deadline from the referral's urgency. The hospital that failed to respond has its performance record degraded, which lowers its future ranking.
2. **Otherwise, if the referral is an emergency** — search for the nearest active hospital within twenty kilometres that has not already been tried, and reassign to it with a fresh fifteen-minute deadline. This is the fallback that prevents an emergency from dying because a short ranked list was exhausted.
3. **Otherwise** — reject the referral automatically with an explanatory reason, so the consultant is told rather than left waiting.

Each outcome emits events to the affected parties: the new hospital is notified of an incoming referral, and the previous hospital and the consultant are notified of the escalation.

THE ESCALATION JOB IS A SINGLE-INSTANCE COMPONENT

The job runs inside the API process. Running two API instances would run the job twice, which could double-escalate a referral. Horizontal scaling therefore requires either extracting the job to a dedicated worker or introducing a distributed lock. This constraint is recorded in the Deployment and Operations Guide alongside the related one-time-password constraint.

6. The Commission Engine

6.1 Why It Is a Single Module

Bill splitting was originally implemented separately in four places. Because the arithmetic appeared correct in each, divergence between them would have produced small, plausible, systematically wrong figures — the hardest class of financial defect to detect. The engine exists to make that impossible: it is the only code that divides a bill, it is pure with no database access, and it is the most heavily tested module in the system with seventeen dedicated test cases.

6.2 The Two Models

CareBridge supports two commission models. The nested model was the original arrangement; the additive model was introduced later at stakeholder request. Both are live, selected per consultant, and a single settlement may contain consultants on either.

6.2.1 Nested model

```
platformDeduction = bill x facility.deductionPercentage
consultantFee      = platformDeduction x consultant.commissionPercentage
platformCharge     = platformDeduction - consultantFee
facilityOwes       = platformDeduction
```

```
Worked example – bill PKR 10,000, deduction 20%, commission 60%
platformDeduction = 10,000 x 20% = 2,000
consultantFee     = 2,000 x 60% = 1,200
platformCharge    = 2,000 - 1,200 = 800
facility keeps    = 10,000 - 2,000 = 8,000
```

The consultant fee is carved out of the platform's deduction. A more generous consultant rate therefore reduces the platform's share and does not increase the facility's liability.

6.2.2 Additive model

```
consultantFee = percentage of bill OR a fixed amount
platformCharge = percentage of bill OR a fixed amount (independently chosen)
facilityOwes   = consultantFee + platformCharge
```

```
Worked example – bill PKR 10,000, consultant fixed PKR 1,500, platform 10%
consultantFee = 1,500 (fixed, independent of bill size)
platformCharge = 10,000 x 10% = 1,000
facility owes  = 1,500 + 1,000 = 2,500
facility keeps = 10,000 - 2,500 = 7,500
```

The two components are independent. The platform charge is never reduced to fund the consultant fee — there is no subsidy in either direction. This was the specific stakeholder requirement that motivated the model.

6.3 The Governing Invariant

TOTAL DEDUCTION ALWAYS EQUALS CONSULTANT FEE PLUS PLATFORM CHARGE

This identity holds under both models, both component types, every combination, and for both hospitals and laboratories. It is asserted explicitly in the test suite. Because it holds, every downstream consumer — settlement totalling, ledgers, earnings views, reports — can aggregate across consultants on different models without knowing which model applies to which. That is what makes mixed settlements work.

6.4 Terms Resolution Order

Both the consultant fee and the platform charge are resolved by searching a defined order and taking the first match.

1. Consultant-facility override
+-- Does this consultant have an override recorded for THIS facility?
| (matched on both facility type and facility identity)
| yes --> use its type and value. STOP.
v no
2. Facility terms
+-- Does the facility record its own charge type and value?
| yes --> use them. STOP.
v no
3. Platform defaults
+-- Use the configured platform default percentage. STOP.
v not set
4. Built-in constants
+-- Deduction 20%, commission 60%.

Figure 6: Resolution of the platform charge for one case

The override tier is what allows an individually negotiated rate between one consultant and one facility, without affecting either party's dealings with anyone else. An override is scoped to the facility it names; the tests assert explicitly that an override for one hospital does not apply at another.

AN OVERRIDE APPLIED TO A CONSULTANT ON THE NESTED MODEL

Where a consultant-facility override exists but the consultant is still on the nested model, the engine preserves the nested consultant fee exactly and substitutes only the platform charge. That single case behaves additively while every other case for the same consultant remains nested. This is intentional, and it is why the override is stored as a distinct tier rather than by mutating the consultant's own terms.

6.5 The Laboratory Variant

A laboratory bill is computed as the gross of the priced tests, less the consultant's discount, giving the amount payable. Under the additive model the engine then iterates the test lines individually, applying the discount factor to each line before charging it — necessary because a laboratory fixed charge applies per test rather than per referral. The consultant's discount is capped at a maximum held on their record.

6.6 Precision and Guards

- **Integer paisa throughout** — every input and output is a whole number of paisa. Rounding occurs once, at a defined point, rather than accumulating across a chain of floating-point operations.
- **Percentages are clamped** — any value outside zero to one hundred is bounded, and a non-numeric value becomes zero rather than propagating as an invalid number.
- **Conversion is guarded at the boundary** — the administrator interface accepts rupees; conversion to paisa rejects negative and non-numeric input.
- **The facility's residual may be negative** — where a fixed charge exceeds a very small bill, the facility keeps a negative amount. The engine reports this faithfully rather than silently clamping it, and the case is covered by an explicit test. It is a commercial configuration question, not an arithmetic one.

7. The Settlement Cycle

7.1 Design Intent

Settlement is deliberately manual and evidence-based. Money moves between the parties outside the platform; the platform records what was agreed, requires documentary evidence at each stage, and requires an administrator to verify that evidence before the next stage proceeds. No stage advances automatically.

7.2 The Five States

State	Meaning	Advanced by
Pending payment	The settlement has been assembled from billed cases. Payment to the platform is due.	The facility uploading a payment receipt.
Pending administrator verification	A receipt has been uploaded and awaits checking.	An administrator verifying it, or rejecting it with a reason that returns it to the facility.
Paid, pending consultant payout	Payment is confirmed. Consultant commissions are due for release.	An administrator recording each payout with its receipt.
Paid, pending consultant verification	Payouts have been recorded and await confirmation by the consultants.	Each consultant confirming receipt.
Completed	All parties have confirmed. The cycle is closed.	Terminal state.

7.3 Accrual and Aggregation

A consultant commission is accrued at the moment a case is billed, not when a settlement is assembled. The accrual carries its own terms snapshot. Assembling a settlement therefore sums accruals that already exist, which is why a settlement may freely mix consultants on different models — the arithmetic was already resolved case by case, and the governing invariant guarantees the totals reconcile.

Billing a case is idempotent: repeating the operation on an already-billed case returns without financial effect, so a duplicate submission cannot create a second accrual.

A SUPERSEDED MODULE REMAINS IN THE CODEBASE

An earlier design credited consultant wallets automatically on billing and released funds once a balance passed a threshold, retaining an initial hold. That module is still present and its threshold settings are still configurable, but it is not connected to the current billing flow. It recomputes commission arithmetic inline rather than through the engine, which makes it the one place in the codebase where the single-calculation-path principle is violated. Its removal is the first recommendation in the Maintenance and Support Guide.

8. Cross-Cutting Design

8.1 Authentication and Authorisation

Authentication issues a short-lived access credential carrying the user identity and role, plus a long-lived renewal credential carrying only the identity. Renewal is rolling — each renewal issues a fresh pair — and is refused for an account that is no longer active, which is the mechanism by which suspending an account takes effect on an existing session.

Authorisation is a guard that compares the caller's role against the roles permitted for the endpoint, with one deliberate exception: the administrator role passes unconditionally. The role listed against an endpoint is therefore the minimum role, not an exclusive one, and every endpoint in the system is administrator-reachable.

TWO CONSEQUENCES OF THE DESIGN

First, because the administrator bypass is unconditional, an administrator account is an unrestricted account; administrator accounts should be few and individually named so that audit entries identify a person. Second, there is no credential revocation list — a renewal credential remains valid for its full lifetime, and the only server-side control is the account status check at renewal. Suspending an account is the effective response to a compromised session.

8.2 Field-Level Encryption

The patient national identity number is encrypted before storage and decrypted on retrieval, transparently at the schema level, so no controller has to remember to apply it. Each value is encrypted with a fresh initialisation vector, so the same identity number stored twice produces different stored values. Values without the expected format pass through unchanged, which allows records that predate encryption to remain readable.

The mode in use provides confidentiality but not authentication — modification of stored ciphertext would not be detected on decryption. An upgrade to an authenticated mode is recommended in the Security and Compliance document. In production the application refuses to start without an encryption key; in development it warns and stores plaintext.

8.3 Notifications

A notification is raised once and fanned out across three channels concurrently — in-application, email, and WhatsApp — with the outcome of each channel independent, so a failure on one does not prevent delivery on the others or fail the operation that raised it. Eighteen event types are defined, each with its own title and message template.

Facility notifications resolve to the facility owner and all its active team members, so an incoming referral reaches everyone able to act on it. Personal data is stripped from the stored notification payload, so the notification collection holds no patient or contact information.

8.4 Real-Time Event Model

Room	Membership	Purpose
user:<id>	One user	Personal notifications
role:<role>	All users of a role	Role-wide broadcast
hospital:<id>	A hospital's owner and team	Inbox, bed, and referral events
consultant:<id>	One consultant	Referral status and settlement events
lab:<id>	A laboratory's owner and team	Laboratory referral events

The client subscribes to nine event types and collapses them into a single cache invalidation after a short delay, so a burst of related events causes one refresh rather than several. The server holds a single socket reference in a registry, which is what allows services and the scheduled job to emit events without an HTTP request in scope.

8.5 Configuration and White-Labeling

Behaviour that a platform operator may reasonably wish to change is held in configuration rather than code: commission defaults, matching weights, the department and keyword catalogue, wallet thresholds, and branding. None requires a code change or redeployment. Branding resolves in two tiers — platform branding applies by default, and a hospital's own branding overrides the name, logo, and primary colour for that hospital's users — applied at runtime as CSS custom properties.

9. Known Design Debt

The following are recorded as design observations rather than defects. Each is understood, none affects correctness of the delivered behaviour, and each is a candidate for a future phase. They are stated here so that a maintaining engineer meets them in documentation rather than in the code.

Observation	Consequence	Recommendation
The superseded wallet crediting module remains present	Commission arithmetic exists in one place outside the engine, violating the single-calculation-path principle.	Remove the module and the settings that belong only to it.
One-time-password state is held in process memory	Codes do not survive a restart, and the backend cannot run as more than one instance.	Move the store to a shared cache before scaling.
The escalation job runs inside the API process	A second API instance would run the job twice.	Extract to a worker or introduce a distributed lock, at the same time as the above.
Hospital and laboratory settlement endpoints have different path shapes	Two identical state machines are reached through differently structured paths, which is a small ongoing cost to every reader.	Align the paths in a future version of the interface.
Client data fetching is mixed between two patterns	Two conventions coexist for the same job.	Migrate the remaining screens to the query cache.
Two screens open their own socket connection	Those users hold two connections where one would do.	Consolidate onto the shared connection.
Two built screens are unreachable	A WhatsApp broadcast console and a public landing page exist but are not linked into the application.	Route them or remove them; leaving built code unreachable is misleading to a maintainer.
No integration test layer	Controllers, routes, the scheduler, and the settlement state machines have no automated regression protection.	Add integration tests over the settlement state machine and the escalation job first, as the highest-value targets.